

Kotlin assignment ITW

20 Easy Challenges (Code Snippets & One-Liners)

Encapsulation, Inheritance & Abstraction

1. **Encapsulation:** Write a class `BankAccount` with a `private var balance: Double`. Provide a public function `deposit(amount: Double)` that safely adds to the balance.
2. **Custom Setter:** Write a class `Thermostat` with a `var temperature: Double`. Give it a `private set` so it can only be modified from within the class.
3. **Inheritance:** Create an `open class Vehicle` with a function `startEngine()`. Create a `Motorcycle` subclass that inherits from it and overrides `startEngine()` to print "Vroom!".
4. **Super Constructor:** Write a parent class `Employee(val baseSalary: Int)`. Write a child class `Manager` that inherits from `Employee` and correctly passes `80000` up to the `super` constructor.
5. **Abstraction:** Write an `abstract class Animal` with an abstract method `makeSound()`. Write a `Cat` class that inherits from it and implements the sound.

Data Classes, Singletons & Sealed Classes

6. **Data Class:** Write a `data class User` that holds an `id (Int)`, `username (String)`, and `isActive (Boolean)`.
7. **The Copycat:** Instantiate a `User` from the previous question. Write one line of code using `.copy()` to create a new user with the exact same ID and username, but set `isActive` to `false`.
8. **Singleton:** Write an `object Database` with a `fun connect()` that prints "Connected to DB".
9. **Companion Object:** Write a `class IDGenerator`. Inside it, create a `companion object` with a variable `currentId` starting at 0, and a function `getNextId()` that increments and returns it.
10. **Sealed Class:** Write a `sealed class ScreenState`. Add an `object Loading` and a `data class Success(val data: String)` that inherit from it.

Lambdas, Collections & Extensions

11. **Lambda Syntax:** Write a variable `val square: (Int) -> Int` that holds a lambda function. Use the `it` keyword to return the square of the input number.
12. **Higher-Order Function:** Write a function `executeTwice(action: () -> Unit)` that simply calls the passed-in `action` lambda two times in a row.
13. **Filter:** Create a `listOf(1, 2, 3, 4, 5)`. Write a one-liner using `.filter { }` to create a new list containing only the even numbers.

14. **Map:** Create a `listOf("apple", "banana")`. Write a one-liner using `.map { }` to create a new list containing the *lengths* of those strings.

15. **Extension Function:** Write an extension function for the `String` class called `addExclamation()` that returns the string with `!"` appended to the end. (e.g., `"Hello".addExclamation()` becomes `"Hello!"`).

Scope Functions & Coroutines

16. **Apply:** Create a standard `class Window` with `var width = 0` and `var height = 0`. Instantiate it and chain an `.apply { }` block to set both dimensions to 1920 and 1080 immediately.

17. **Let:** Given `val input: String? = "Kotlin"`, write a safe `.let { }` block using the `?` operator that prints the length of the string *only* if it is not null.

18. **Suspend:** Write a `suspend fun fetchWeather(): String` that uses `delay()` to wait 1000ms, then returns `"Sunny"`.

19. **Launch:** Inside a `fun main() = runBlocking { }` block, use `launch { }` to start a background coroutine that prints `"Background task finished"`.

20. **Async:** Inside a `runBlocking` block, write code using `async { }` to calculate `5 + 5` and then use `.await()` to print the result.

5 Medium Challenges (Connecting the Dots)

21. The Map/Filter Combo:

You have a list of products: `val inventory = listOf(Product("Laptop", 1000.0), Product("Mouse", 25.0), Product("Keyboard", 60.0))`.

Write a chained collection operation that first uses `.map { }` to apply a 10% discount to all prices, and then uses `.filter { }` to return only the products that still cost *more than* \$50.

22. The Exhaustive State Machine:

Expand on your `ScreenState` sealed class from Question 10 by adding an `Error(val errorMsg: String)` state. Then, write a function `fun renderUI(state: ScreenState)` that uses an exhaustive `when` expression to print a specific message for all three possible states.

23. The Concurrent Downloader:

Write two suspend functions: `fetchUserData()` (delays 1000ms, returns `"Alice"`) and `fetchAppConfig()` (delays 2000ms, returns `"Dark Mode"`). In your `main` function, use `async` to fetch them both *at the exact same time*. Await both results and print them. Total execution time must be ~2 seconds, not 3.

24. The Builder Pattern (Scope Functions):

Write a function `fun buildFile(filename: String): File`. Inside the function, instantiate a `File` object. Chain an `.apply { }` block to set its `permissions` to `"Read/Write"`. Then chain an `.also { }` block to log `"File $filename created"` to the console. Finally, return the configured object.

25. Polymorphic HOF:

Create an **abstract class** `Shape` with an **abstract fun** `calculateArea(): Double`. Create `Circle` and `Square` subclasses. Write a Higher-Order Function **fun** `processShapes(shapes: List<Shape>, action: (Double) -> Unit)`. Iterate through the list, calculate each area, and pass the result into the `action` lambda.

1 Hard Challenge (The System Architect)

26. The E-Commerce Checkout Pipeline

Write a complete, executable Kotlin program in a single file that simulates a concurrent checkout system.

Requirements:

1. Create a **data class** `CartItem(val name: String, val price: Double, val quantity: Int)`.
2. Write an **Extension Function** on `List<CartItem>` called `calculateTotal()` that multiplies the price by the quantity for each item and returns the sum as a `Double`.
3. Create a **Sealed Class** called `CheckoutResult` with three states: `Processing`, `Success(val receiptId: String)`, and `Failed(val reason: String)`.
4. Create a **Singleton object** `PaymentProcessor` containing a **suspend fun** `processPayment(cart: List<CartItem>): CheckoutResult`.
 - Inside this function, delay for 2 seconds to simulate a network call.
 - Use your extension function to get the cart total.
 - If the total is greater than \$1000, return `Failed("Insufficient funds for large transaction")`.
 - Otherwise, return `Success("TXN-${(1000..9999).random()}")`.
5. In your `main` function (using `runBlocking`), create two separate carts (one cheap, one expensive). Use `launch` to process them concurrently through the `PaymentProcessor`. Use an exhaustive `when` statement to print the final outcome of each cart!